ParaTest

The objective of ParaTest is to support parallel testing in PHPUnit. Provided you have well-written PHPUnit tests, you can drop `paratest` in your project and start using it with no additional bootstrap or configurations!

Benefits:

- Zero configuration. After the installation, run with `vendor/bin/paratest` to parallelize by TestCase or `vendor/bin/paratest --functional` to parallelize by Test. That's it!
- Code Coverage report combining. Run your tests in N parallel processes and all the code coverage output will be combined into one report.

Installation

To install with composer run the following command:

```
composer require --dev brianium/paratest
```

Versions

Only the latest version of PHPUnit is supported, and thus only the latest version of ParaTest is actively maintained.

This is because of the following reasons:

1. To reduce bugs, code duplication and incompatibilities with PHPUnit, from version 5 ParaTest heavily relies on PHPUnit `@internal` classes
2. The fast pace both PHP and PHPUnit have taken recently adds too much maintenance burden, which we can only afford for the latest versions to stay up-to-date

Usage

After installation, the binary can be found at `vendor/bin/paratest`. Run it with `--help` option to see a complete list of the available options.

Test token

The `TEST_TOKEN` environment variable is guaranteed to have a value that is different from every other currently running test. This is useful to e.g. use a different database for each test:

```php
if (getenv('TEST_TOKEN') !== false) {  // Using ParaTest
    $dbname = 'testdb_' . getenv('TEST_TOKEN');
} else {
    $dbname = 'testdb';
}
```

A `UNIQUE_TEST_TOKEN` environment variable is also available and guaranteed to have a value that is unique both per run and per process.

Code coverage

The cache is always warmed up by ParaTest before executing the test suite.

PCOV

If you have installed `pcov` but need to enable it only while running tests, you have to pass thru the needed PHP binary option:

```
php -d pcov.enabled=1 vendor/bin/paratest --passthru-php="'-d' 'pcov.enabled=1'"
```

xDebug

If you have `xDebug` installed, activating it by the environment variable is enough to have it running even in the subprocesses:

```
XDEBUG_MODE=coverage vendor/bin/paratest
```

Initial setup for all tests

Because ParaTest runs multiple processes in parallel, each with their own instance of the PHP interpreter, techniques used to perform an initialization step exactly once for each test work different from PHPUnit. The following pattern will not work as expected - run the initialization exactly once - and instead run the initialization once per process:

```php
private static bool $initialized = false;

public function setUp(): void
{
    if (! self::$initialized) {
         self::initialize();
         self::$initialized = true;
    }
}
```

This is because static variables persist during the execution of a single process. In parallel testing each process has a separate instance of `$initialized`. You can use the following pattern to ensure your initialization runs exactly once for the entire test invocation:

```php
static bool $initialized = false;

public function setUp(): void
{
    if (! self::$initialized) {
        // We utilize the filesystem as shared mutable state to coordinate between processes
        touch('/tmp/test-initialization-lock-file');
        $lockFile = fopen('/tmp/test-initialization-lock-file', 'r');

        // Attempt to get an exclusive lock - first process wins
        if (flock($lockFile, LOCK_EX | LOCK_NB)) {
            // Since we are the single process that has an exclusive lock, we run the initi
            self::initialize();
        } else {
            // If no exclusive lock is available, block until the first process is done wit
            flock($lockFile, LOCK_SH);
        }

        self::$initialized = true;
    }
}
```

Troubleshooting

If you run into problems with `paratest`, try to get more information about the issue by enabling debug output via `--verbose --debug`.

When a sub-process fails, the originating command is given in the output and can then be copy-pasted in the terminal to be run and debugged. All internal commands run with `--printer [...]\NullPhpunitPrinter` which silence the original PHPUnit output: during a debugging run remove that option to restore the output and see what PHPUnit is doing.

Caveats

1. Constants, static methods, static variables and everything exposed by test classes consumed by other test classes (including Reflection) are not supported. This is due to a limitation of the current implementation of `WrapperRunner` and how PHPUnit searches for classes. The fix is to put shared code into classes which are not tests *themselves*.

Integration with PHPStorm

ParaTest provides a dedicated binary to work with PHPStorm; follow these steps to have ParaTest working within it:

1. Be sure you have PHPUnit already configured in PHPStorm: https://www.jetbrains.com/help/phpstorm/using-phpunit-framework.html#php_test_frameworks_phpu
2. Go to `Run` -> `Edit configurations...`
3. Select `Add new Configuration`, select the `PHPUnit` type and name it `ParaTest`
4. In the `Command Line` -> `Interpreter options` add `./vendor/bin/paratest_for_phpstorm`
5. Any additional ParaTest options you want to pass to ParaTest should go within the `Test runner` -> `Test runner options` section

You should now have a `ParaTest` run within your configurations list. It should natively work with the `Rerun failed tests` and `Toggle auto-test` buttons of the `Run` overlay.

Run with Coverage

Coverage with one of the available coverage engines must already be configured in PHPStorm and working when running tests sequentially in order for the helper binary to correctly handle code coverage

For Contributors: testing ParaTest itself

Before creating a Pull Request be sure to run all the necessary checks with `make` command.